Watcher — Implementation Roadmap

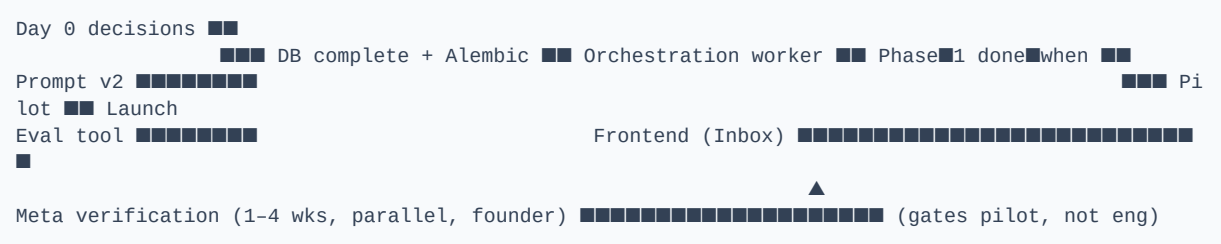
Status: Planning artifact. Sequences every track/point from the architecture review into a build order with owners, dependencies, and a Day-0 board. Updated 2026-06-10.

Where we are: ~45% of the MVP is built — 248 backend tests green, full conversation layer ported, receptionist reply path wired as fourth orchestrator outcome. What's missing is the concrete adapters (LLM providers, pywa, gspread, httpx), intent taxonomy unification, REST API, and the entire frontend.

The one thing that gates the calendar: Meta WABA verification is 1–4 weeks of *waiting*. It does not block engineering (use the dev test number), but it blocks the first real pilot. **Start it on Day 0.**

---

0. Critical path (what determines the launch date)



The longest *engineering* pole is **Prompt v2 + Eval tool** (the AI-Engineer's two highest-leverage, least-done items). The longest *calendar* pole is **Meta verification**. They run in parallel.

---

1. Day 0 — What we get on TODAY

Three lanes start in parallel. None of these are blocked.

Lane A — Founder / decisions (no code)

- [ ] **Start Meta Business verification** — create Business Account, submit legal entity docs, request WABA + dev test number. *(Longest calendar pole — every day counts.)*
- [ ] **Create Anthropic + OpenAI accounts**, generate API keys, note rate limits.
- [ ] **Answer the Decision Gate** (§2 below) — these are choices, not code; all can be made today.
- [ ] **Confirm pilot LOIs** — lock ≥1 of the 2–3 (Phase 0 Track 1).

Lane B — Engineering, unblocked (can start the moment decisions in §2 land)

- [ ] **DB schema completion** — author the remaining 8 SQLAlchemy models + **Alembic init + first migration**. *(Models/migrations can be written against the Postgres dialect with no live DB.)*
- [ ] **Eval golden set** — begin authoring the 50 JSONL examples (10/intent, EN/AR/mixed). *(Pure content work; needs the taxonomy decision from §2, which is a 10-minute call.)*
- [ ] **Per-tenant policy slice** — move the hardcoded thresholds (bands 0.85/0.5, identity 0.92/0.75, TTL, retries) into a per-tenant settings object. *(No external deps; closes the customizability gap from the last review.)*
- [ ] **Housekeeping** — delete the stale `claude/nifty-johnson-f89mpz` branch (empty `cowork/` noise).

Lane C — Frontend, unblocked

[ ] **Scaffold the Next.js 15 app** + implement DESIGN-SPEC.md tokens/typography (Inter/Cairo, Lucide). *(DESIGN-SPEC is done; the shell + static views need no backend.)*

Net: by end of Day 0 we can have Meta verification in flight, all decisions locked, and four engineering/build streams (DB, eval content, tenant-policy, frontend shell) actively moving.

---

2. Decision Gate (Track 0) — must close before Sprint 1

All are founder decisions; recommended defaults in **bold** so we can proceed if you concur.

#	Decision	Recommended default	Unblocks
D8-a	First-pass / escalation / fallback **model IDs**	**Haiku** → **Sonnet** → **GPT-4o-mini** (confirm exact IDs)	LLM providers, eval
—	**Intent taxonomy**	**Adopt role-guide's 6 intents + 3 record types as enums**	Prompt, schema, eval
—	**Band thresholds**	**0.85 high / 0.5 medium** (align repo's 0.60 → 0.5)	Router, classifier
D9-a	Identity dedup scope	**Cache-only v1** (addendum wins over flowchart's CRM roundtrip)	Identity, router
D2-a	Auth provider	**Clerk** (SaaS)	Frontend auth, API
D3-a	ASR provider	**Whisper API** (SaaS)	Media concrete impl
D13-a	Eval in CI	**Recorded fixtures in CI; live key nightly**	Eval CI gate
—	Hosting	**Render** (fastest) or AWS	CD deploy, Alembic target
—	Schema shape	**Flat (addendum §4)**, pin the enums above	DB, classifier

---

3. Sprint plan (agent-assisted; ~1 sprint ≈ 1 week)

Each numbered item is one PR-sized slice. Items map 1:1 to the review's Tracks 1–3.

Sprint 1 — Phase-1 foundations (DB · Prompt · Eval)

Goal: the AI pipeline is real and measurable.

#	Slice	Track	Depends on	Owner
1	Remaining 8 DB tables + Alembic init + first migration + RLS notes	T1-1	Hosting (for target), schema decision	Backend

2	**Prompt v2** — embed confidence rubric verbatim + 10+ few-shot (EN/AR/mixed) + pinned intent/record enums	T1-2	Taxonomy, thresholds	AI Eng
3	**Eval tool** (`packages/eval`) — golden set (50) + CLI runner + 5 metrics + HTML/JSON + wire the CI gate	T1-3	Taxonomy; D13-a	AI Eng
4	**Concrete LLM providers** — Anthropic + OpenAI behind the `LLMProvider` seam	T1-4	D8-a, keys	AI Eng
5	Per-tenant policy/settings (thresholds, TTLs)	(gap fix)	—	Backend

Gate: eval **baseline accuracy report** locked (T1-6). Every later change measured against it.

Sprint 2 — The product comes together (Orchestrator · Inbox)

Goal: a real message becomes a routed, audited record, visible in an inbox.

#	Slice	Track	Depends on	Owner
6	**Orchestration worker** — queue consumer → source-exclusion → media → history fetch → classify → identity → rules → band routing → destination → audit → inbox_item	T1-5	Sprints 1 (DB, providers)	Backend
7	Real queue wiring (BackgroundTasks now; arq/Redis seam)	T1-5	#6	Backend
8	**REST API** for control page (inbox/sources/destinations/rules) + `PATCH /classifications/{id}` correction endpoint	T2-10	#1, #6	Backend
9	**Inbox view** (the critical path screen) against REST API	T2-11	#8, Lane C shell	Frontend

Gate: Phase-1 done-when (T1-7) — real message to the dev test number → classified Postgres row in <10s.

Sprint 3 — Connectors & human-in-the-loop

#	Slice	Track	Depends on	Owner
10	**pywa outbound** — control-chat pings using the built signed buttons + reply route in webhook	T2-8	#6	Backend
11	**Google Sheets connector** (gspread + service account) + real **httpx** webhook transport behind the delivery port	T2-9	#6	Backend
12	**Sources · Destinations · Rules** views	T2-11	#8	Frontend
13	**Auth (Clerk)** + tenancy binding	T2-12	D2-a	Full-stack

Gate: end-to-end on the test number — message → inbox → one-click route to Sheets/webhook, audited.

Sprint 4 — Deploy & harden

#	Slice	Track	Depends on	Owner
14	Wire CD deploy step to chosen host; **staging** env up; Alembic runs in deploy	T2-13	Hosting	DevOps
15	Media concrete impl (Whisper + vision) behind ports	(Phase-1.5)	D3-a, keys	AI Eng
16	Admin/Eval dashboard API + view	T3-16	#3, #8	Full-stack
17	Hardening: retries/backoff, dead-letter surfacing, error budgets	—	Sprints 2–3	Backend

Gate: product deployable to staging; DESIGN-SPEC Definition-of-Done screens functional.

Sprint 5+ — Pilot & launch (calendar-gated by Meta approval)

#	Slice	Track
18	Onboard pilot on **their** WABA; 2 weeks observation, daily inbox review	T3-14
19	Correction logging → passive-learning loop; golden set 50 → 150	T3-15
20	Rules live in prod; weekly dedup sweep job; Arabic verified on real traffic	T3-16
21	Billing (Stripe), soft-cap behavior; onboard 2 paying pilots	T3-17

4. Milestone gates (Definition of Done per stage)

Milestone	Criteria	Target
M0 — Decisions locked	§2 gate fully answered; Meta verification submitted; keys in hand	Day 0–1
M1 — AI pipeline measurable	Prompt v2 + eval baseline report committed	End Sprint 1
M2 — Phase-1 done-when	Real test-number message → classified Postgres row <10s	End Sprint 2
M3 — End-to-end deployable	Inbox + one-click route to a destination + audit, on staging	End Sprint 3–4
M4 — Pilot live	Bot on real client WABA, watching ≥1 chat, founder triaging daily	Meta-gated
M5 — MVP shipped	Pilot renews after 30 days, no manual intervention (flowchart Part 6)	Pilot + 30d

5. Owner lanes (who drives what)

Lane	Owns
Founder	Meta verification, decisions, pilot LOIs, billing, Arabic QA
AI Engineer	Prompt v2, eval tool, LLM providers, model tiering, passive learning, media ASR
Backend	DB/Alembic, orchestrator, REST API, connectors, pywa, identity/rules wiring
Frontend	Next.js control page (Inbox-first), admin/eval dashboard
DevOps	CD deploy, staging/prod infra, RLS/tenancy, monitoring

(Solo + AI agents: same lanes, sequenced rather than parallel — Inbox before other views, eval before prompt iteration.)

6. Risk watch (carry forward from the audit)

Risk	Mitigation	Status
Meta verification slips	Build on dev test number; start Day 0	Open
Confidence miscalibration	Rubric in prompt + eval calibration metric every CI run	Sprint 1
Arabic accuracy drift	Per-language eval metric; production verify Phase 4	Sprint 1 / pilot
AWS Bedrock MENA availability	Research call before first regulated sale	Watch (Phase 5)

Self-hosted pricing	2–3 GCC conversations during pilot	Watch (Phase 5)

---

7. Progress log — RESUME HERE (updated 2026-08-14)

> A fresh session has no chat memory; this section + `DECISIONS.md` + `AGENTS.md` + `docs/HANDOFF.md` are the handoff.

Repo state: PR #13 on `claude/file-review-planning-dcyb2g` — 5 commits (Items 1.1, 1.3, 2.1, 2.2, 2.3). **248 backend tests green**; `ruff` + `ruff format` + `mypy --strict` all clean.

Done (all behind ports, fully unit-tested):

- Schemas · ingestion/webhook · classifier tiering (slices 1–3) — *merged*
- Domain: rules · identity · control-chat · destinations · audit — *merged*
- Persistence: full §4 DB schema (11 tables) + Alembic initial migration — *merged*
- Media pipeline — *merged*
- Lane-A decisions applied: taxonomy enums, band 0.5, `core/policy.py::TenantPolicy` — *merged*
- Eval golden-set seed (8 examples) + frontend design tokens — *merged*
- **Orchestration worker** (`apps/api/orchestration/`) — end-to-end routing keystone — *merged*
- **Eval runner** (`packages/eval`) — 5 metrics + recorded-fixture CI gate (baseline **0.875**) — *merged*
- **Queue/worker wiring** (`apps/api/orchestration/queue.py`) — BackgroundTasks consumer — *merged*
- **De-WhatsApp the core** (Item 1.1) — channel-neutral renames, `KNOWN_LEAKS = {}`, Alembic 002 — *in PR #13*
- **Python 3.13** (Item 1.3) — 7 version pins bumped — *in PR #13*
- **Conversation layer** (Item 2.1) — 7 new tables (Alembic 003), ORM models, `ConversationRepository` — *in PR #13*
- **Receptionist reply path** (Item 2.2) — tool registry, `receptionist.py`, `ChannelSender` protocol — *in PR #13*
- **Autonomy gate wiring** (Item 2.3) — `RECEPTIONIST_REPLY` fourth outcome, `decide_autonomy()` before rules — *in PR #13*

Known design gap: `IntentType` (classification enum) and vocabulary intents (receptionist taxonomy) are separate. The autonomy gate returns `hand_off` for any intent not in the vocabulary, so the receptionist path only fires when the classifier produces a vocabulary-recognized intent. Unifying the two taxonomies is prerequisite to the receptionist handling real traffic.

Next up: 1. **Intent taxonomy unification** — merge `IntentType` with vocabulary intents so `decide_autonomy()` recognises classified intents and the receptionist fires on real messages. 2. **Concrete LLM providers** (Anthropic/OpenAI) against the `LLMProvider` seam — *needs API keys*. 3. **Grow golden set 8 → 50** (10/intent, EN/AR/mixed) → lock the real baseline accuracy number. 4. **DB-backed** `MessageLoader` for `orchestration/queue.py` + wire into webhook route. 5. **Real** `ChannelSender` for WhatsApp (Meta Cloud API outbound). 6. **REST API** for control page + Inbox view.

Build-loop working agreement (match this style): Python-only backend; each slice = ports + Pydantic v2, fully unit-tested; `ruff` + `ruff format` + `mypy --strict` + `pytest` all green before commit; line-length 100; one slice per commit; open a PR when asked; CI installs deps explicitly in `ci.yml`.

External / founder carry-over: start Meta WABA verification (longest pole), Anthropic + OpenAI keys, Render Postgres URL, confirm ≥1 pilot LOI.